

1. Общая логика приложения

Запуск приложения

↓

Сезонная видео-заставка

↓

Главный экран

↓

Проверка тревожных контактов

↓

Нажатие «Я живу»

↓

Сохранение отметки

↓

Открытие открытки

↓

Отправка открытки тревожным контактам

Приложение состоит из нескольких отдельных экранов и сервисных функций.

Есть 3 иконки приложения для:

- Light mode
- Dark mode
- Tinted mode

2. Точка входа — MorningHelloApp.swift

Это главный файл запуска приложения. Он показывает: IntroVideoView, а после завершения видео открывает: ContentView

Видео-заставка сама по себе не должна открывать открытку. После видео пользователь всегда попадает на главный экран.

3. IntroVideoView.swift

Этот экран отвечает только за заставку. Он:

- определяет текущий сезон;
- выбирает соответствующий MP4;
- воспроизводит его без элементов управления;
- после завершения сообщает приложению, что заставка закончилась.

Сопоставление:

Март—май → spring_intro

Июнь—июль → summer_intro

Август → august_intro

Сентябрь—ноябрь → autumn_intro

Декабрь—февраль → winter_intro

Видео лежат в папке проекта `Videos` и входят в Target Membership приложения.

4. ContentView.swift

Это центральный файл приложения. Сейчас он выполняет несколько ролей одновременно:

- показывает главный экран;
- показывает открытку;
- управляет отметкой «Я живу»;
- выбирает изображение и поздравительный текст;
- открывает формы контактов, праздников и профиля;
- запускает локальные уведомления;
- подготавливает отправку открытки;
- проверяет обязательное наличие тревожных контактов.

Фактически `ContentView` сейчас является главным контроллером приложения.

Примерно такие состояния используются внутри `ContentView`:

```
@State private var showContacts = false
@State private var showHolidaySettings = false
@State private var showProfile = false
@State private var showPostcard = false

@State private var hasCheckedIn = false
@State private var lastCheckInDate: Date?

@State private var hasEmergencyContacts = false

@State private var showMessageComposer = false
@State private var showContactForWhatsApp = false
@State private var emergencyContactsForSharing:
[EmergencyContact] = []
```

Их назначение:

- `showContacts` — открыть экран тревожных контактов;
- `showHolidaySettings` — открыть выбор праздников;
- `showProfile` — открыть профиль;
- `showPostcard` — показать открытку;
- `hasCheckedIn` — пользователь уже отметил, что с ним всё хорошо;
- `lastCheckInDate` — время последнего нажатия;
- `hasEmergencyContacts` — есть ли хотя бы один тревожный контакт;
- `showMessageComposer` — открыть iMessage/SMS;
- `showContactForWhatsApp` — показать выбор контакта для WhatsApp.

Разделение логики отметки и открытки - важное архитектурное решение. **hasCheckedIn** отвечает за сам факт отметки. **showPostcard** отвечает только за отображение открытки. Открытка открывается только после нового нажатия на кнопку.

Все открытки хранятся в формате JPEG с разрешением 880 на 1168 пикселей с разрешением 72 пикселей на дюйм.

Реализована автоматическая смена фоновой иллюстрации главного экрана в зависимости от времени суток:

- Утро
- День
- Вечер
- ночь

Главный экран содержит:

Доброе утро
Как ты сегодня?

Далее поведение зависит от тревожных контактов.

Если контактов нет

Кнопка «Я живу» не показывается. Вместо неё пользователь видит сообщение:

Добавьте тревожный контакт

и кнопку открытия `EmergencyContactsView`.

Если контакт есть

Появляется кнопка:

Я живу

или:

Я в порядке

если отметка ещё действует.

Ниже выводится предупреждение:

Если ты не нажмёшь кнопку в течение 48 часов — мы сообщим близким

Также на главном экране находятся кнопки:

Контакты
Праздники
Профиль

Логика кнопки «Я живу»:

Если 24-часовая блокировка ещё не действует:

- сохраняется текущая дата;
- `hasCheckedIn` становится `true`;
- дата записывается в `UserDefaults`;
- планируется локальное напоминание.

После этого открывается открытка.

Если пользователь уже отмечался, новая дата не записывается, но открытка всё равно может открыться.

5. postcardScreen

Это экран открытки. Он содержит:

- выбранное изображение;
- выбранный текст;
- кнопку «Домой»;
- кнопку отправки.

Кнопка «Домой»:

```
showPostcard = false
```

Она возвращает на главный экран, но не стирает отметку.

В `ContentView` используются два вычисляемых свойства:

```
var smartImage: String  
var smartPhrase: String
```

Они определяют, какую картинку и какой текст показать сегодня. Логика построена на приоритетах. Текущий порядок такой:

1. День рождения пользователя
2. Государственные, религиозные и тематические праздники
3. Понедельничные открытки - 48 штук
4. Декабрьские открытки - 26 штук
5. Шаббат (если включены еврейские праздники)

6. Воскресные открытки (если включены православные или католические праздники) - 48 штук
7. Февральские открытки - 18 штук
8. Обычные сезонные открытки

Благодаря этому:

- понедельник всегда имеет больший приоритет, чем февраль или декабрь;
- воскресенье имеет больший приоритет, чем февраль;
- февраль отображается только если не сработал ни один из предыдущих приоритетов.

То есть более важное событие перекрывает обычную открытку.

Модель праздничной открытки. Используется структура:

```
struct HolidayContent {  
    let images: [String]  
    let phrases: [String]  
    let category: String  
}
```

Она хранит:

- названия изображений в Assets;
- тексты поздравлений;
- категорию праздника.

Категории:

```
neutral  
orthodox  
catholic  
jewish
```

Нейтральные праздники показываются всегда. Религиозные — только если соответствующая настройка включена.

Изображение остаётся постоянным в течение суток, текст поздравления также остаётся неизменным в течение суток. Это исключает ситуацию, когда при повторном открытии открытки пользователь видит другой текст.

6. Настройки праздников — HolidaySettingsView.swift

Форма позволяет включить или отключить:

Православные
Католические

Еврейские

Нейтральные праздники показываются всегда.

Эта форма также содержит:

- объяснение логики праздников;
- раскрываемый список всех праздников;
- прокрутку;
- кнопку закрытия.

7. Воскресные открытки

Для воскресений есть отдельный набор из 48 открыток:

Sunday_1
Sunday_2
...

и соответствующие фразы. Открытка выбирается детерминированно, а не случайно.

Функция:

```
stableDailyIndex(...)
```

использует дату и `salt`.

Поэтому в течение одного воскресенья:

- картинка не меняется;
- текст не меняется;
- повторные нажатия показывают ту же открытку.

8. Декабрьские открытки

В декабре используется отдельный набор из 26 открыток. Особенности:

- используются все подготовленные декабрьские открытки;
- открытки выбираются автоматически;
- специальные открытки (Шаббат, воскресенье и праздничные дни) имеют более высокий приоритет;
- оставшиеся дни месяца заполняются декабрьскими открытками без повторений в течение периода.

Такой подход позволяет легко масштабировать систему на любые месяцы года.

9. Февральские открытки

Создана функция для набора из 18 открыток

```
februaryImage ( )
```

которая автоматически выбирает открытку:

February_1
February_2
...
February_28
February_29

в зависимости от текущего дня месяца.

Особенности:

- 29 февраля используется только в високосные годы;
- выбор не случайный;
- открытка полностью соответствует календарному дню.

10. Шабатные открытки

Создан набор:

Shabbat_1
...
Shabbat_11

Шабатная открытка доступна:

с ПЯТНИЦЫ 18:00
до СУББОТЫ 18:00

Только если включены еврейские праздники.

Функции:

`shabbatStartDate(for:)`
`currentShabbatContent(for:)`

определяют:

- находится ли текущее время внутри шабата;
- какую открытку показывать.

Для одного шабата выбирается одна и та же картинка и одна и та же фраза.

11. Понедельничные открытки с кофе

В приложении реализован отдельный набор мотивационных открыток из 48 штук, которые показываются пользователю по понедельникам после нажатия кнопки «Я живу». Реализована отдельная логика отображения специальных открыток по понедельникам.

Зимний период

Период:

- 1 сентября — 28 (29) февраля. Используется набор изображений:

MondayCold_1

...

MondayCold_24

Тематика:

- чашка кофе;
- уют;
- осенняя и зимняя атмосфера;
- начало рабочей недели.

Тёплый период

Период:

- 1 марта — 31 августа. Используется набор изображений:

MondayWarm_1

...

MondayWarm_24

Тематика:

- летнее утро;
- кофе;
- солнечная атмосфера;
- позитивное начало недели.

Принцип выбора

Создан единый механизм `mondayCoffeeContent()`.

Он автоматически:

1. определяет текущую дату;
2. выбирает нужный сезон (Warm/Cold);
3. возвращает соответствующий набор изображений и текстов.

Это позволило отказаться от отдельных функций для каждого сезона и централизовать логику понедельников.

Все изображения хранятся локально в **Assets**, поэтому открытки доступны без подключения к интернету. Выбор сезона и конкретной открытки выполняется непосредственно на устройстве пользователя.

Приоритет праздничных и персональных открыток остаётся выше обычной понедельничной открытки. Поэтому, когда понедельник совпадает с праздником или днём рождения пользователя, приложение показывает более приоритетный контент.

12. Дни рождения

Дата рождения пользователя хранится в профиле:

```
profile_birth_day  
profile_birth_month
```

Имя хранится:

```
profile_display_name
```

Функция:

```
birthdayContent()
```

проверяет:

- заполнены ли день и месяц;
- совпадают ли они с сегодняшней датой.

Если совпадают, возвращает:

```
holiday_birthday
```

и персональный текст:

<имя из профиля пользователя>, с днём рождения!

Если имя не заполнено, используется обычное поздравление без имени. День рождения имеет самый высокий приоритет среди открыток.

В приложении MorningHello добавляется отдельный пользовательский сценарий “Поздравить с днём рождения”.

Пользовательский сценарий

Главный экран

↓

Поздравить с днём рождения

↓

Выбрать открытку

↓

Выбрать получателя

↓

Контакт №1
или

Контакт №2

или

Любой контакт

↓

Отправить изображение и текст

Сценарий не зависит от автоматического показа ежедневной открытки и открывается вручную с главного экрана.

Пользователь может:

1. Выбрать одну из десяти открыток ко дню рождения.
2. Прочитать соответствующее поздравление.
3. Отправить открытку:
 - тревожному контакту №1;
 - тревожному контакту №2;
 - любому контакту через системное меню iPhone.

Кнопка размещается после основного блока проверки состояния пользователя и перед кнопками:

Контакты

Праздники

Профиль

Поздравить с днём рождения

Создаётся отдельный SwiftUI-файл:

`BirthdayGreetingView.swift`

Его задача — полностью управлять интерфейсом выбора и отправки поздравительной открытки.

Каждому изображению соответствует отдельная фраза. Количество изображений и количество фраз должно совпадать.

Чтобы `ShareSheet` был доступен не только из `ContentView`, но и из `BirthdayGreetingView`, он выносится в отдельный файл: `ShareSheet.swift`

`BirthdayGreetingView` отвечает за:

выбор открытки

перелистывание

выбор получателя

подготовку текста и изображения

открытие `iMessage`

открытие системного меню

`ShareSheet` отвечает за показ `UIActivityViewController`

13. Пасхальная неделя

Логика православной Пасхи - реализована отдельная категория

Православный

которая охватывает всю Светлую седмицу.

Создана функция

`orthodoxHolyWeekContent ( )`

Она:

- определяет дату Пасхи;
- вычисляет диапазон:

Пасха

↓

Пасхальный понедельник

↓

...

↓

Светлая суббота

для каждого дня недели.

Пасхальные открытки

Создан отдельный набор изображений

`orthodox_easter_week_1`

...

`orthodox_easter_week_7`

(и может быть расширен).

Пасхальные поздравления

Добавлены специальные поздравления, начинающиеся словами:

- «Христос воскрес!»
- «Со Светлой Пасхой!»
- «Со светлым праздником Пасхи!»

Фразы выбираются автоматически по стабильному ежедневному индексу.

Приоритет

Пасхальная неделя является частью категории

Православный

поэтому автоматически имеет высокий приоритет среди праздничных открыток.

14. Профиль — ProfileView.swift

Экран содержит:

- имя пользователя, до 50 символов;
- день рождения;
- месяц рождения;
- статус подписки;
- кнопку обновления подписки.

Данные сохраняются локально через `@AppStorage`.

Подписка должна в дальнейшем поступать через `StoreKit`.

Пока статус может отображаться как:

Подписка не активна

15. Тревожные контакты — EmergencyContactsView.swift

Экран позволяет добавлять близких. Модель контакта такая:

```
struct EmergencyContact: Codable, Identifiable {  
    let id: UUID  
    var name: String  
    var surname: String  
    var phone: String  
    var email: String  
}
```

Контакты сохраняются в `UserDefaults` в виде JSON.

Ключ:

"emergency_contacts"

`ContentView` читает этот массив через `checkEmergencyContacts()` и определяет, можно ли показать кнопку «Я живу».

Логика проверки формы разделена по типам данных. Отдельно проверяются:

- имя;
- фамилия;
- международный номер телефона;
- адрес электронной почты.

Это позволяет показывать пользователю понятные сообщения именно о том поле, которое заполнено неверно.

Для формы добавления тревожных контактов была реализована собственная система проверки номера телефона.

При сохранении контакта введённый номер сначала нормализуется: удаляются пробелы, тире и скобки, после чего выполняется проверка соответствия международному формату.

Проверяются следующие условия:

- номер начинается со знака «+»;
- после знака «+» содержится только цифровая последовательность;
- количество цифр соответствует требованиям международного стандарта (от 8 до 15 цифр).

Если хотя бы одно условие нарушено, сохранение контакта не выполняется. Пользователю выводится всплывающее сообщение с описанием ошибки и примером правильного ввода номера.

Такой подход позволяет исключить сохранение некорректных телефонных номеров и уменьшает вероятность ошибок при использовании функции экстренного вызова.

Поле **E-mail** переведено в разряд обязательных данных.

Причины изменения:

- возможность автоматической отправки уведомлений по электронной почте;
- резервный канал связи при отсутствии доставки через WhatsApp или iMessage;
- единый формат хранения контактов.

Добавлена полноценная проверка корректности адреса электронной почты перед сохранением контакта.

16. Отправка открытки (кнопка “Поделиться”)

Кнопка отправки открывает:

1) WhatsApp

Если установлен WhatsApp:

`whatsapp://send?phone=...&text=...`

открывает чат конкретного контакта. Для каждого сохранённого контакта отображается отдельная кнопка. Например

WhatsApp: Иван Иванов

В Info проекта добавлено:

Queried URL Schemes
whatsapp

Важно:

- WhatsApp нельзя заставить отправить сообщение автоматически;
- пользователь сам нажимает «Отправить»;
- изображение открытки не всегда можно прикрепить через URL-схему;
- текст подставляется автоматически.

2) iMessage / SMS

Используется:

MFMessageComposeViewController

через отдельный файл:

MessageComposerView.swift

Он может заранее добавить:

- номера всех тревожных контактов;
- текст сообщения;
- изображение текущей открытки.

Добавлено понятие **Display Name пользователя**, которое используется как единая точка получения имени владельца приложения.

Имя пользователя планируется использовать в нескольких подсистемах:

- отображение имени на открытках;
- формирование персональных поздравлений;
- формирование сообщений при отправке открыток через WhatsApp и iMessage;
- дальнейшее использование в автоматических уведомлениях и e-mail сообщениях.

Для хранения имени используется единый ключ в **UserDefaults**, что позволяет обращаться к нему из любого модуля приложения без дублирования логики.

Сообщение состоит из двух независимых частей:

1. персональное приветствие;
2. текст текущей открытки.

Формат сообщения:

Имя пользователя желает вам доброго утра!

Текст открытки.

Если имя пользователя отсутствует, используется нейтральное приветствие.

Такое разделение позволяет в дальнейшем менять только приветствие, не изменяя тексты самих открыток.

3) Поделиться с любым контактом

Пользователь может выбрать:

- Контакты;
- WhatsApp;
- Telegram;
- Почту;
- AirDrop;
- Messages;
- любое другое установленное приложение.

При этом передаются одновременно:

- изображение открытки;
- поздравительный текст.

4) Отмена

Диалог можно закрыть без выбора действия.

17. Архитектура модуля «Обратная связь»

Модуль «Обратная связь» позволяет пользователю напрямую связаться с разработчиком MorningHello из экрана профиля.

Основные сценарии:

- предложить новый праздник;
- сообщить о проблеме в приложении;
- отправить обычное сообщение разработчику.

Модуль работает через стандартные системные инструменты iOS:

- `MFMailComposeViewController` для электронной почты;
- `MFMessageComposeViewController` для SMS и iMessage.

Сообщение формируется заранее, но пользователь может изменить текст перед отправкой.

Кнопка «Обратная связь» размещена в `ProfileView` под блоком подписки. Под кнопкой отображается микро-текст: “Я читаю все сообщения лично”.

Экран профиля состоит из независимых секций:

```
nameSection
birthdaySection
subscriptionSection
feedbackSection
```

Экран FeedbackView

FeedbackView является отдельным SwiftUI-экраном. Он содержит:

- заголовок «Хочешь написать мне?»;
- три варианта обращения;
- выбор способа отправки;
- обработку недоступности Mail или Messages;
- системные формы отправки.

Варианты обращения:



Предложить праздник



Сообщить о проблеме



Просто написать

Предзаполненные сообщения “Предложение праздника”

Тема письма:

Предложение праздника

Текст:

Я хотел(а) предложить новый праздник.

Название:

Дата:

Почему он важен:

Предзаполненные сообщения “Сообщение о проблеме”

Тема письма:

Ошибка в приложении

Текст:

Опишите проблему:

Что произошло:

На каком экране:

Предзаполненные сообщения “Обычное сообщение”

Тема письма:

Сообщение разработчику

Текст:

Здравствуйте!

После выбора типа обращения открывается `confirmationDialog`. Пользователь может выбрать:

Отправить по электронной почте
Отправить через Сообщения
Отмена

На симуляторе или неподготовленном устройстве Mail и Messages могут быть недоступны. В этом случае системный контроллер не открывается. Пользователю показывается `Alert` с объяснением:

- почтовый аккаунт не настроен;
- устройство не поддерживает SMS или iMessage;
- можно выбрать другой способ отправки.

Файловая структура

`ProfileView.swift`
`FeedbackView.swift`

В `FeedbackView.swift` находятся:

`FeedbackType`
`FeedbackView`
`FeedbackMailComposer`
`FeedbackMessageComposer`

Это изолирует функциональность обратной связи от основного экрана профиля.

Пользовательский сценарий

Профиль

↓

Обратная связь

↓

Выбор типа обращения

↓

Выбор Mail или Messages

↓

Открытие предзаполненного сообщения

↓
Редактирование пользователем

↓
Отправка или отмена

Для добавления нового типа обращения достаточно расширить:

`FeedbackType`

и добавить:

`title`
`icon`
`emailSubject`
`messageBody`

18. Уведомления

Используется:

`UNUserNotificationCenter`

Функция:

`requestNotificationPermission()`

запрашивает разрешение.

Функция:

`scheduleCheckInReminder()`

создаёт локальное напоминание.

Сейчас уведомление локальное — оно приходит самому пользователю. Оно не может самостоятельно уведомить тревожные контакты.

Определена архитектура сценария:

- пользователь не подтверждает своё состояние;
- спустя 48 часов запускается сценарий тревоги;
- приложение предлагает выбрать тревожный контакт;
- автоматически формируется сообщение для WhatsApp или iMessage.

Архитектура рассчитана на последующее расширение, включая отправку сообщений по e-mail.

Используется системный `confirmationDialog`, который:

- загружает список сохранённых контактов;
- отображает имя каждого контакта;
- позволяет выбрать получателя сообщения;

- после выбора запускает соответствующий канал отправки.

В главный объект приложения интегрирован `CheckInNotificationManager`.

Теперь при запуске приложения автоматически:

- запрашивается разрешение на уведомления;
- регистрируются действия уведомлений;
- инициализируется система ежедневных check-in;
- создаётся основа для дальнейшего контроля периода в 48 часов.

18. Backend JSON

Добавлена архитектура подготовки данных для будущего сервера. Созданы модели

`EmergencyContactPayload`

`LastCheckInPayload`

`UserStatusPayload`

Добавлены функции

`createBackendPayload()`

`createBackendJSON()`

Формируется JSON, содержащий

- `appId`;
- версию приложения;
- статус пользователя;
- время последней отметки;
- выбранный интервал проверки;
- список тревожных контактов.

Последний сформированный JSON сохраняется локально в

`UserDefaults`

для последующей отправки после подключения серверной части.

Добавлена система хранения времени последней отметки пользователя. Используются

`lastCheckInDate`

и

`lastCheckInKey`

При каждом подтверждении «Со мной всё хорошо» сохраняются:

- дата;
- время.

При следующем запуске приложения автоматически восстанавливается последнее состояние.

Что хранится локально

Через `UserDefaults` и `@AppStorage` сейчас сохраняются:

Имя пользователя
День рождения
Месяц рождения
Тревожные контакты
Последняя отметка
Выбор религиозных праздников
Некоторые настройки интерфейса

Эти данные хранятся только на устройстве. Облачного аккаунта и серверной базы пока нет.

Что пока не реализовано полностью

Самые важные незавершённые части:

Автоматическое уведомление через 48 часов

Приложение на iPhone не может надёжно само отправить email, SMS или WhatsApp, когда оно закрыто.

Для этого понадобится сервер:

iPhone
↓
сохраняет дату отметки на сервере
↓
сервер проверяет просрочку
↓
сервер отправляет email или SMS

StoreKit

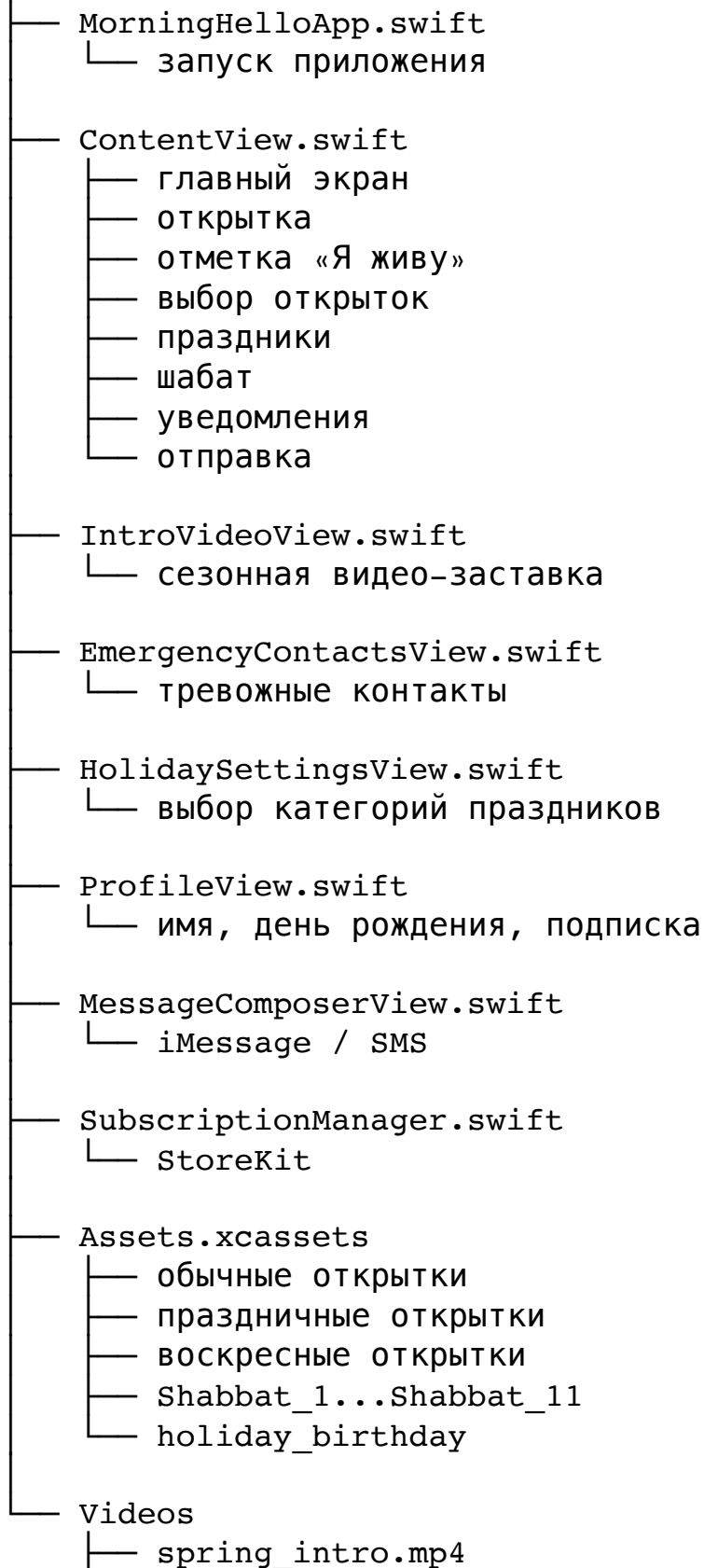
Экран подписки есть, но нужно закончить реальную покупку и проверку подписки.

Профиль в облаке

Сейчас профиль локальный. После удаления приложения данные исчезнут.

Текущая структура файлов

MorningHello



```
|— summer_intro.mp4
|— autumn_intro.mp4
|— winter_intro.mp4
```

Главная архитектурная проблема сейчас

`ContentView.swift` стал слишком большим.

В нём одновременно находятся:

- интерфейс;
- хранение данных;
- выбор праздников;
- уведомления;
- отправка;
- календарные расчёты;
- выбор открыток.

Пока приложение работает, но перед публикацией лучше постепенно разделить его.

Рекомендуемая будущая структура:

```
ContentView
HomeView
PostcardView
HolidayService
ShabbatService
BirthdayService
CheckInManager
EmergencyContactsStore
NotificationManager
ShareManager
SubscriptionManager
```

Это снизит риск ошибок с фигурными скобками, дублирующими функциями и областями видимости.